Table of Contents Table of Content

1. **Introduction 3**
   - Problem description 3
   - Motivations..... 3
   - Project objectives..... 3

2. **System requirements.....4**
   - **Functional requirements..... 4**
   - **Non-functional requirements……4**

3. **System Architecture.....4**
   - Overall system design…… 4
   - Client-server architecture diagram…… 4
   - Database schema (ERD) ……5
   - Authentication mechanism ……6
   - Security measures used……..6

4. **Implementation Details**
   - Technologies used……. 7
   - Frontend implementation……..7
   - Backend implementation ……..7
   - Database design ……..8
   - Challenges faced ……8
   - Assumptions made……8

5. **Security Considerations**
   - How authentication is handled ……8
   - How passwords are stored ……..
   - How data is protected ……..9
   - Access control implementation……9

6. **Testing & Results**
   - Screenshots of the application ……10
   - Explanation of major features working ……11
   - Description of any incomplete features (if any) ……..12

7. Future Work

- Possible improvement……..**12**
- Scalability consideration……**12**

1.Introduction

A. Problem description

Many people and businesses today need online services such as graphic design, programming, writing, computer repair, and digital marketing. These services are in high demand, but connecting trustworthy clients with skilled gig workers is still a big challenge.

One of the main problems is **payment insecurity**. Freelancers often worry about whether they will be paid fairly and on time. Another issue is **fraud and fake profiles**, where dishonest users pretend to be someone they are not. There is also **poor dispute resolution**, meaning when conflicts happen, they are not solved quickly or fairly. Finally, **communication barriers** make it hard for clients and freelancers to understand each other clearly, which slows down projects.

B. Motivation

The motivation behind this project is to study how gig worker platforms operate and learn from them. By analyzing their system architecture, we can see how they are built and how they function.

We also want to identify their strengths and weaknesses. This will help us understand what works well and what needs improvement. With this knowledge, we can propose better solutions that make freelancing platforms more secure, more reliable, and easier to use.

C. Project objectives

The project has several clear objectives. First, we want to analyze the **functional and non-functional requirements** of gig worker platforms. This means looking at what the system must do and how well it should perform.

Second, we will examine the **system architecture** to understand how different parts of the platform work together. Third, we will evaluate the **security mechanisms** to see how safe the platform is for both clients and freelancers. Finally, we will identify **system weaknesses** so that we can suggest improvements for better service delivery.

2. System Requirements

A. Functional requirements

The system must allow **user registration**, so both clients and gig workers can create accounts. It should also support **login and authentication** to keep accounts secure.

Clients must be able to **post jobs**, and freelancers should be able to apply for them. The system should enable **messaging between users**, making communication smooth and direct. It must also support **secure payments**, ensuring freelancers are paid safely. Finally, the system should allow **reviews and ratings**, so users can build trust and reputation.

B. **Non-functional requirements**

The platform should be **highly available**, meaning it is always accessible when users need it. It must provide **fast response times**, so actions like posting jobs or sending messages happen quickly.

Data privacy is very important, so the system must **protect user information**. It should also be **scalable**, meaning it can grow to support millions of users without breaking down. Lastly, the platform must have a **user-friendly interface**, making it simple and enjoyable for people to use.

3. System Architecture

A. Overall system design

The platform is built using a **web-based client-server architecture**. This means users can access the system through a website or a mobile app, while the server handles all the heavy tasks like storing data, processing requests, and managing security. The client side (website or app) focuses on giving users a smooth and friendly experience, while the server side ensures everything runs safely and efficiently.

B. Client-Server Architecture Diagram

In this design, the **client** (user's device) sends requests such as logging in, posting jobs, or sending messages. The **server** receives these requests, processes them, and sends back the right response. This separation makes the system more reliable and easier to maintain. It also allow the

platform to grow and support many users at the same time.

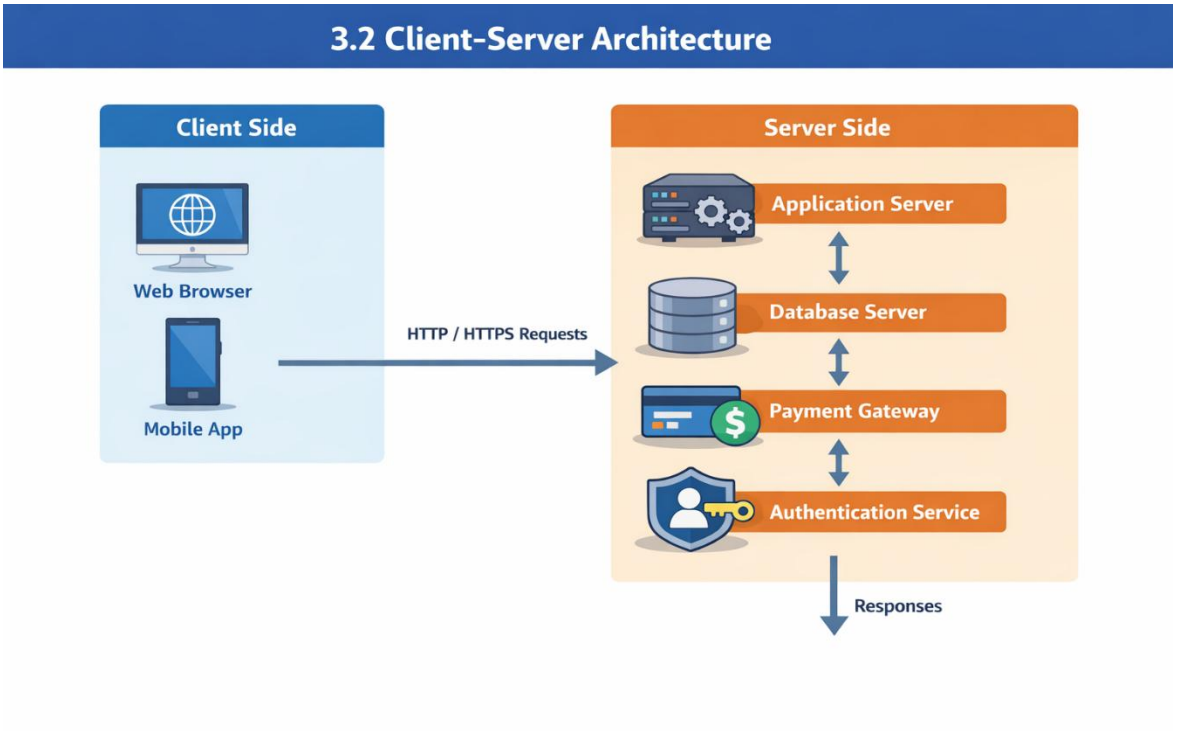

Fig 1.Client – server architecture

C. Database schema (ERD overview)

The database is the heart of the system, storing all important information. The main entities include:

a) **Users**: Each user has a unique ID, name, email, and role (client or gig worker).

b) **Jobs**: Jobs are posted by clients and include details like description and budget.

c) **Proposals**: Freelancers send proposals with bid amounts for specific jobs.

d) **Contracts**: Once a proposal is accepted, a contract is created between the client and gig worker.

e) **Payments**: Payments are linked to contracts, showing the amount and status.

f) **Reviews**: After work is completed, clients can leave ratings and comments.

The relationships are simple and logical. One client can post many jobs. Each job can receive multiple proposals. A contract connects one client and one gig worker. A contract can also have multiple payments, depending on milestones.

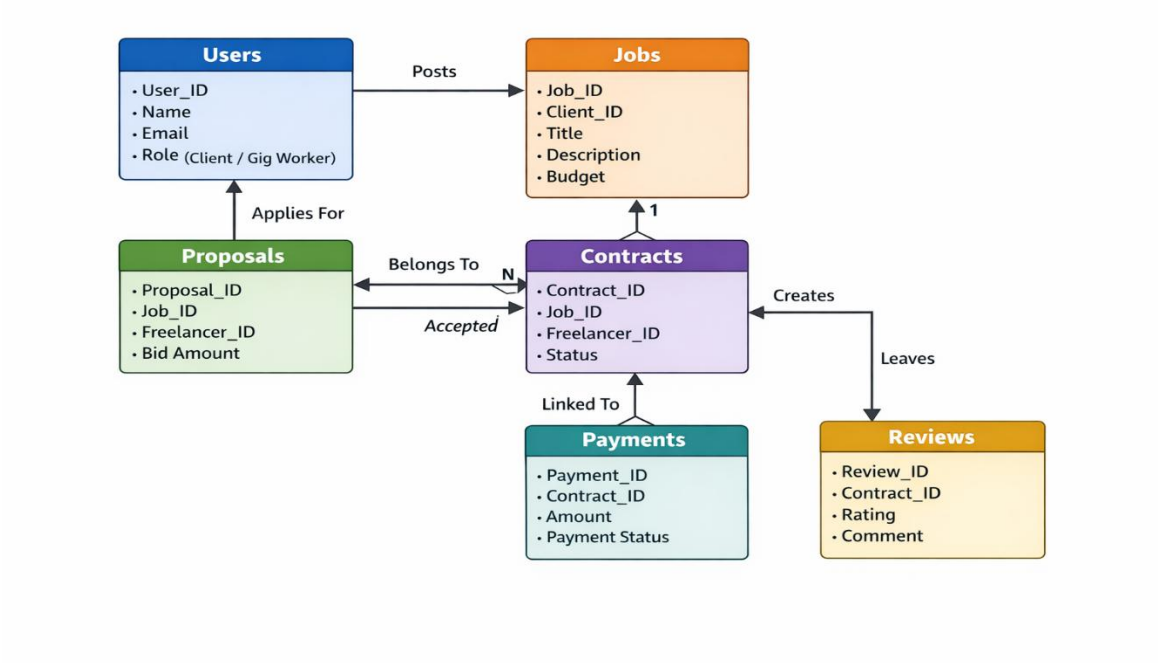

Fig 2. Database schema

D. Authentication mechanism

To keep accounts safe, the system uses several authentication methods. Users can log in with their **email, mobile number, and password**. For convenience, optional logins through **Google or Facebook** are available.

For extra security, **two-factor authentication** can be enabled, adding another layer of protection. The system also uses **session-based authentication**, which keeps users logged in securely while they use the platform.

E. Security measures used

Security is a top priority for this platform. All communication between users and the server is protected with **HTTPS encryption**, ensuring data cannot be intercepted.

Payments are handled through an **escrow system**, which holds money safely until the work is completed. This protects both clients and freelancers. **Identity verification** helps reduce fake profiles and fraud. Finally, **activity monitoring** keeps track of unusual behavior, making it easier to detect and prevent misuse.

4. Implementation details

A. Technologies used

To build this platform, we used modern and reliable technologies.

a) **Frontend**: The user interface is created using HTML, CSS, and JavaScript. These tools help design a clean and responsive layout that works well on both computers and mobile devices.

b) **Backend**: The server side is powered by Node.js and Express.js. These technologies handle user requests, manage data, and run the core logic of the platform.

c) **Database**: We use Firebase to store and manage data. It's fast, secure, and easy to scale.

d) **Payment Integration**: PayPal is used to process payments safely and efficiently.

B. Frontend implementation

The frontend focuses on giving users a smooth experience.

a) The interface is **responsive**, meaning it adjusts to different screen sizes.

b) There are **dashboards** for both clients and gig workers, showing relevant information.

c) Clients can use a **job posting form** to describe their needs.

d) Freelancers have a **proposal submission interface** to send their offers.

e) A **messaging interface** allows users to chat and stay updated on their projects.

C. Backend implementation

The backend is built using **RESTful APIs**, which help different parts of the system communicate smoothly.

a) **Authentication middleware** ensures only authorized users can access their accounts.

b) **Payment processing integration** connects with PayPal to handle transactions.

c) The backend also manages **business logic**, such as posting jobs, accepting proposals, and creating contracts.

D. Database design

The database is designed to be efficient and secure.

a) It follows a **relational structure**, meaning data is organized in tables with clear relationships.

b) **Foreign key constraints** help link related data, like jobs to users or payments to contracts.

c) We use **indexed search** to make job filtering fast and accurate, helping users find what they need quickly.

E. Challenges faced

While building the platform, we faced several challenges:

a) **Handling payment disputes** was tricky, especially when clients and freelancers disagreed.

b) **Preventing fake accounts** required strong verification methods.

c) **Ensuring secure transactions** meant adding layers of protection to keep user data and money safe.

F. Assumptions made

To make the system work smoothly, we made a few basic assumptions:

a) Users have **internet access** to use the platform.

b) **Payment gateways** like PayPal are reliable and available.

c) Users provide **accurate information** when signing up and posting jobs.

5. Security Considerations

A. How authentication is handled

To protect user accounts, the platform uses strong authentication methods. **Passwords are encrypted**, meaning they are stored in a way that makes them unreadable to anyone. When users log in, the system creates **secure login sessions** to keep their access safe while they use the platform.

We also use **token validation**, which helps confirm that each user request is genuine and not coming from a hacker or fake source. These tokens expire after a short time, adding another layer of protection.

B. How passwords are stored

Passwords are never stored in plain text. Instead, we use **password hashing**, which turns each password into a scrambled code. Even if someone accesses the database, they won't be able to read the actual password.

To make it even safer, we add **salted hashes** — random data mixed into the password before hashing. This makes it harder for attackers to guess passwords using common techniques.

C. How data is protected

All data shared between users and the server is protected using **SSL/TLS encryption**. This means that messages, login details, and payments are scrambled during transmission so no one can intercept them.

We also apply **database access restrictions**, so only authorized parts of the system can read or change sensitive data. In case of unexpected issues, we perform **regular backups** to make sure no important data is lost.

D. Access control implementation

To make sure users only see what they're allowed to, we use **role-based access control**. This means clients, gig workers, and admins each have different permissions based on their role.

We also apply **permission-based resource access**, which ensures that users can only interact with jobs, messages, or payments that belong to them. This keeps the platform secure and fair for everyone.

6. Testing & Results

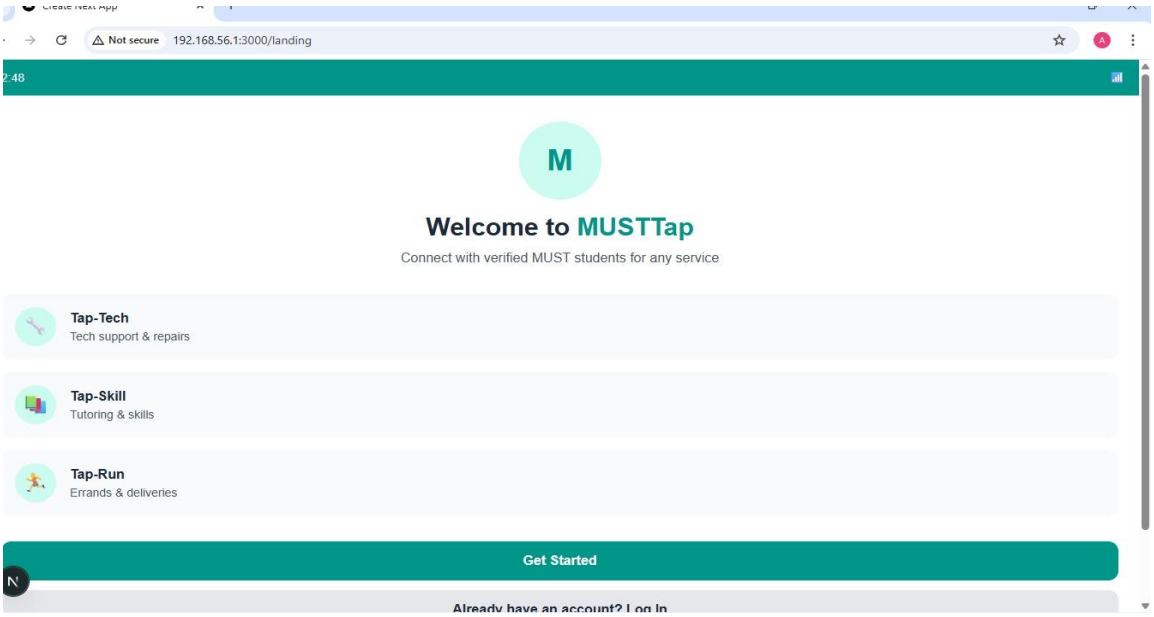

Fig 3.Dashboard

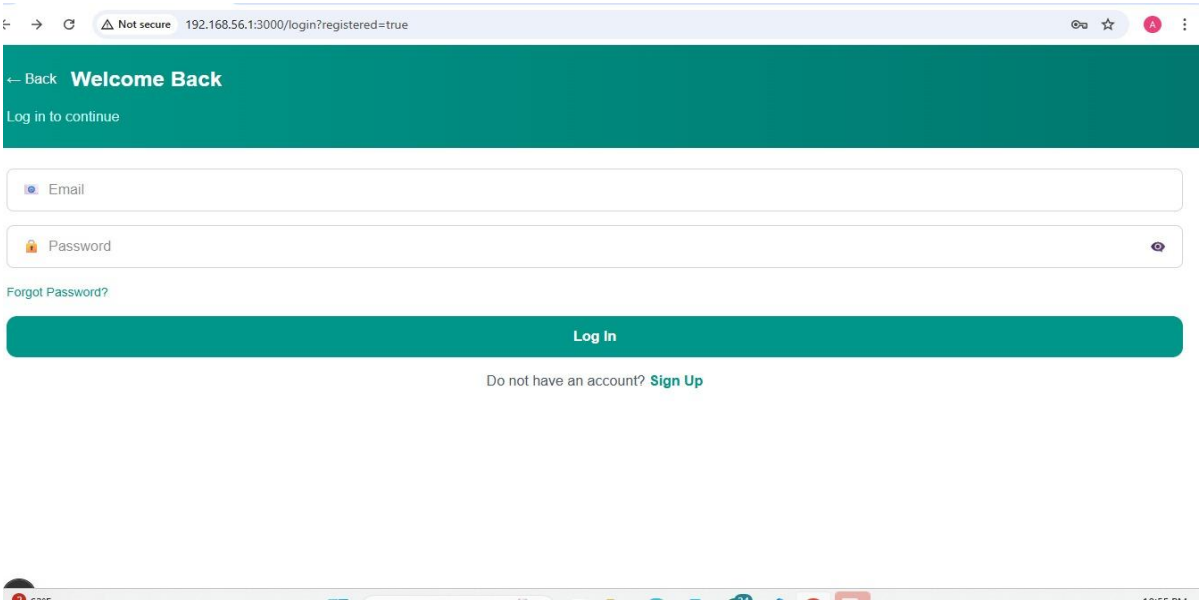

Fig 4. Login page

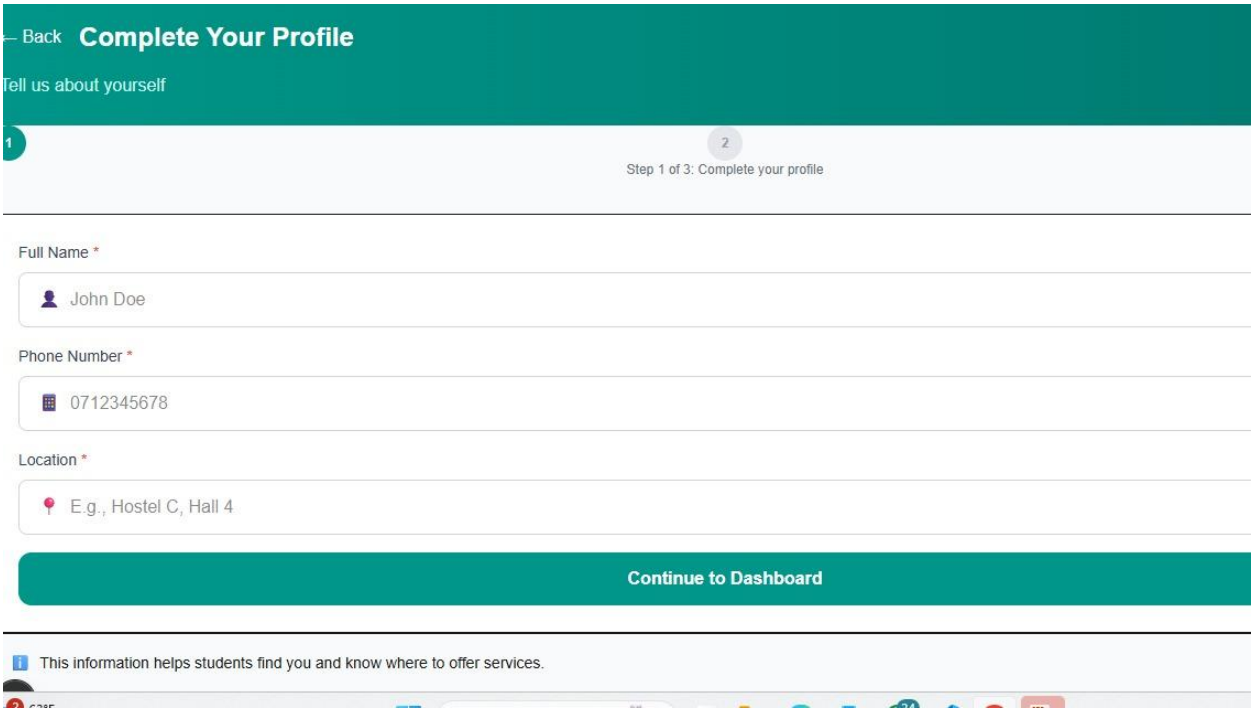

Fig 5. Registration page

B. **Explanation of major features working**

Several core features were tested successfully:

a) **Job Posting**: When a client posts a job, the system creates a new record in the database. This ensures that the job is visible to freelancers.

b) **Proposal System**: Freelancers can submit proposals, which are linked to specific jobs. This connects gig workers directly to opportunities.

c) **Escrow Payment**: Payments are held securely in escrow until the work is completed. This protects both clients and freelancers.

d) **Messaging**: Real-time communication between users works smoothly, allowing quick updates and collaboration.

C. **Incomplete features**

a) Some advanced features are not yet fully implemented:

b) **AI Job Matching**: The system does not yet automatically recommend jobs to freelancers based on their skills.

c) **Automated Fraud Detection**: While basic verification exists, a full fraud detection system is still under development.

d) **Mobile Offline Functionality**: The app currently requires internet access and does not support offline use.

7. Future Work

A. Possible Improvements

a) There are several areas where the platform can grow:

b) **AI-Based Freelancer Recommendation**: Using artificial intelligence to suggest the best freelancers for each job will save time and improve matches.

c) **Blockchain Payment Integration**: Adding blockchain technology could make payments even more secure and transparent.

d) **Improved Dispute Resolution Automation**: Automating conflict resolution will help solve disputes faster and more fairly.

B. Scalability Considerations

To handle growth and millions of users, the system must be scalable:

a) **Load Balancing**: Distributing traffic across multiple servers will keep the platform fast and reliable.

b) **Cloud Hosting**: Hosting on cloud services will allow flexible scaling and reduce downtime.

c) **Microservices Architecture**: Breaking the system into smaller services will make it easier to maintain, upgrade, and expand.